FitPick

Digital Wardrobe & Outfit Planner

Project Report

Software Engineering · Final Project Submission (Part C)

Author **Nithilan A (nithi0307)**

Repository **github.com/nithi0307/fitpick**

Stack **React 19 + Vite** · **FastAPI** · **PostgreSQL 16** · **Docker**

Contents

1. Problem statement
2. User stories
3. System architecture and system design
4. Design of tests
5. Appendix A — Software Requirements Specification
6. Appendix B — Data Flow Diagrams
7. Appendix C — Entity Relationship Diagram
8. Appendix D — UML diagrams
9. Appendix E — Code listing and GitHub link

1. Problem statement

1.1 The problem

Most people own far more clothes than they actually wear. Items get pushed to the back of a wardrobe and forgotten, near-duplicates are bought because nobody can remember what is already owned, and the outfit that worked well last month is never written down anywhere. The result is a daily decision that takes longer than it should and a wardrobe that is used at a fraction of its value.

The underlying difficulty is that a wardrobe has no **index**. A person can only see the few garments at the front of the rail, so there is no way to search it, no way to filter it by occasion or season, and no way to record a combination that worked. Existing shopping apps are built to sell new clothes, not to organise clothes already owned.

FitPick addresses this by giving the wardrobe a digital index. Every garment is recorded once with its category, colour, season, brand, size and price. From then on the collection can be searched, filtered and sorted in seconds, garments can be combined into named outfits that are saved for reuse, and the application reports what is actually owned and how often it is worn.

1.2 Who it is for

Persona	Situation	What FitPick gives them
Priya, 20 College student	A moderate wardrobe on a tight budget. Repeats the same few outfits and has bought duplicates by mistake.	A complete view of what she already owns, so she stops re-buying and starts re-combining.
Arjun, 29 Working professional	Needs formal and smart-casual looks ready for the week and has no time to decide each morning.	Saved outfits tagged by occasion, so getting dressed becomes choosing from a short list.
Meera, 24 Fashion enthusiast	A large wardrobe she enjoys experimenting with, but no record of what she has tried.	An outfit builder, favourites, and statistics about categories, colours and cost per wear.

1.3 Vision and objectives

> **Vision**
> To make wardrobe management simple and organised, so that every user can see what they own, plan outfits easily, and get more value from the clothes they already have.

The objectives that follow from this vision are:

1. Record a garment with enough detail to find it again, in under thirty seconds.
2. Make any garment findable by name, brand, category or season in a single interaction.
3. Let a user assemble and save an outfit from existing garments, and reopen it later.
4. Report honest statistics about the wardrobe — totals, favourites, most worn, value, wear rate.
5. Run identically on any machine so the project can be demonstrated and marked reliably.

1.4 Scope

In scope	Out of scope for this version
Recording, editing and deleting clothing items	AI or machine-learning outfit recommendation
Searching, filtering and sorting the wardrobe	Weather-based or calendar-based suggestions
Building, saving and favouriting outfits	Social features — sharing, following, comments
Dashboard and wardrobe analytics	Shopping, price tracking or retailer integration
Profile screen and account details	Real authentication, sessions and multi-user accounts
Containerised local deployment	Cloud hosting, CI/CD pipeline, mobile applications

1.5 Constraints and assumptions

- **Constraint —** an academic project built by one developer against a fixed deadline, so the technology stack was kept deliberately small.
- **Constraint —** must run locally with free, open tooling; no paid cloud services.
- **Constraint —** no real authentication in this version, so the application is single-user. The database schema already carries a `user_id` on every record so that multi-user support is an addition rather than a rewrite.
- **Assumption —** users are willing to spend some initial effort entering their wardrobe, because the payoff is repeated daily.
- **Assumption —** photographs are optional. Text details alone are enough to make the wardrobe useful, so the application illustrates each garment from its category and colour.
- **Assumption —** users access FitPick from a laptop or phone browser; no native application is needed.

1.6 Success metrics

Metric	Target
Time to record a first clothing item after opening the app	under 2 minutes
Time to assemble and save an outfit	under 1 minute
Wardrobe search response for a 500-item wardrobe	under 1 second
Test users rating the interface “easy to use” or better	80%

2. User stories

Requirements were captured as user stories, written as GitHub issues and prioritised with **MoSCoW** on a project board called *FitPick Development Board*. Fifteen stories were placed in **Must Have** — they define a usable product — and ten in **Should Have**. The Could Have and Won't Have columns were left empty on purpose: anything that did not serve the vision was kept out of the backlog entirely rather than parked in it.

2.1 Must Have

The **Status** column states honestly what exists today. “Full” means the story works end to end through the interface; “API” means the backend endpoint and its tests exist but no screen calls it yet; “UI only” means the screen exists but there is no backend behind it.

#	As a user I want to...	...so that	Status
1	register an account	my wardrobe is private to me	UI only
2	log in	I can reach my own wardrobe	UI only
3	log out	I can leave the app safely on a shared computer	UI only
4	add a clothing item	my wardrobe is recorded in one place	Full
5	edit a clothing item	I can correct details I entered wrongly	API
6	delete a clothing item	items I no longer own stop cluttering the list	API
7	view my whole wardrobe	I can see everything I own at once	Full
8	search my clothing	I can find one garment without scrolling	Full
9	filter my clothing	I can narrow the wardrobe to what is relevant	Full
10	create an outfit	I can reuse a combination that works	Full
11	edit an outfit	I can adjust a saved look	Partial
12	delete an outfit	outfits I no longer wear are removed	API
13	save a favourite outfit	my best looks are easy to find	Partial
14	view a dashboard	I get an overview without hunting for it	Full
15	update my profile	my account details stay correct	UI only

2.2 Should Have

#	As a user I want to...	...so that	Status
16	organise clothing into categories	the wardrobe has a structure I recognise	Full
17	tag items and outfits by occasion	I can dress for the day's plan	Full
18	see recently added clothing	I notice what I have just logged	Full
19	see wardrobe statistics	I understand what I actually own	Full
20	upload a photo of an item	I recognise the garment visually	UI only
21	favourite individual clothing items	my go-to pieces are one click away	Full
22	view the details of one item	I can check size, brand and notes	API
23	sort my clothing	I can see newest, most worn or most expensive first	Full
24	view my outfit history	I can return to looks I built before	Partial
25	change my password	my account stays secure	UI only

2.3 Delivery summary

Status	Count	Meaning
Full	12	Working end to end from the interface against the API and database

Status	Count	Meaning
Partial	3	Core behaviour works; one part of the story is still missing
API only	4	Endpoint implemented and tested, but no screen calls it yet
UI only	6	Screen exists; the supporting backend is listed as future work
Total	25	15 Must Have and 10 Should Have

2.4 A story in full

Each story carries acceptance criteria. Story 4 is shown in full as an example of how the backlog items were written:

Story 4 — Add Clothing Item

As a FitPick user **I want to** add a clothing item with its details **so that** my wardrobe is recorded in one place.

Acceptance criteria

1. The form captures name, category, subcategory, colour, season, brand, size, price, tags and notes.
2. Name, category and colour are required; everything else is optional.
3. The category must be one of the six defined categories, and an unknown value is rejected.
4. On save the item appears immediately in My Wardrobe and in the dashboard totals.
5. The user is returned to My Wardrobe after a successful save.

Criteria 3 and 4 are each covered by an automated test — `test_rejects_unknown_category` and `test_dashboard_updates_after_adding_an_item`. The full traceability from stories to tests is in Section 4.5.

3. System architecture and system design

3.1 Architecture style

FitPick uses a **layered (three-tier) architecture**. The **presentation tier** is a React single-page application; the **application tier** is a FastAPI REST service holding all business rules; the **data tier** is PostgreSQL. Calls travel downwards only — the browser talks to the API, the API talks to the database, and no tier skips a level or calls upwards. All three run as separate containers defined in `docker-compose.yml`.

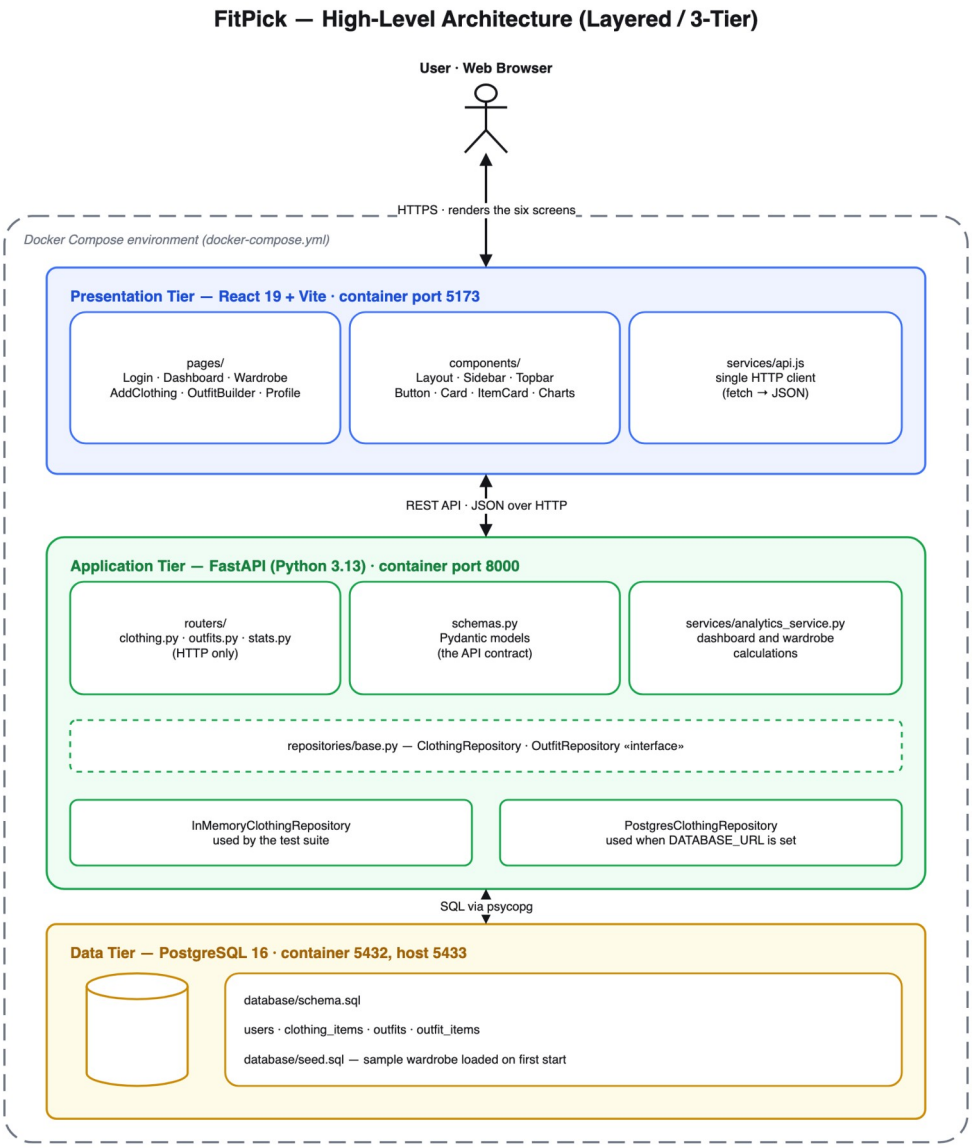


Figure 1 — FitPick high-level architecture (editable source: docs/design/architecture/fitpick-architecture.drawio)

3.2 Why this style was chosen

- **It fits the problem.** FitPick is a CRUD application over a small relational model. A layered design is the standard, proven shape for that, and it avoids the operational cost of micro-services for a system that has one developer and one database.
- **It enforces separation of concerns.** The UI never writes SQL and the database never knows React exists, so each tier can be understood, tested and replaced on its own.
- **It supports independent evolution.** The React app can be restyled, the API can gain endpoints and the schema can be tuned without any of those changes rippling outwards.

- **It is simple to build and explain.** One command starts the whole system, and the single diagram above describes it completely.
- **It leaves room to grow.** Authentication, image storage or outfit suggestions are added as new modules inside the existing tiers, not as a restructure.

3.3 Module responsibilities

Module	Responsibility	Must not
`frontend/src/pages/`	One file per screen; holds screen state and arranges components	know the backend URL or build HTTP requests
`frontend/src/components/`	Reusable presentation — Layout, Sidebar, Topbar, Button, Card, ItemCard, Charts	know which screen is rendering it
`frontend/src/services/api.js`	The only module that talks to the backend; returns plain objects	contain screen logic or formatting
`backend/routers/`	Translate HTTP to repository and service calls, and set status codes	contain storage logic or calculations
`backend/schemas.py`	Pydantic models that define the API contract and validate input	know how data is stored
`backend/services/`	Business calculations for the dashboard and analytics	perform I/O or know about HTTP
`backend/repositories/`	Storage behind one interface, with in-memory and PostgreSQL implementations	know about HTTP or business rules
`database/`	Schema and seed data	contain application logic

3.4 Design principles in the code

Abstraction

Each layer exposes a small, stable interface and hides how it works. Pages never call `fetch`; they call named functions such as `getClothingItems()` and receive plain objects. Routers never touch storage; they call a repository that offers six methods. Because the routers depend only on those method names, FitPick can run against an in-memory store or PostgreSQL with no change to any endpoint:

backend/repositories/__init__.py — one place decides which storage is used

```
def _build() -> tuple[ClothingRepository, OutfitRepository]:
    dsn = os.environ.get("DATABASE_URL")
    if dsn:
        from repositories.postgres_repository import (
            PostgresClothingRepository,
            PostgresOutfitRepository,
        )
        return PostgresClothingRepository(dsn), PostgresOutfitRepository(dsn)

    clothing = InMemoryClothingRepository()
    return clothing, InMemoryOutfitRepository(clothing)
```

Modularity

The codebase is a set of small files with one job each: six page files matching the six designed screens, seven shared components, one API client, three routers, one schema module, one service and one repository per entity. Adding a feature means adding a module, not editing existing ones — a new backend feature is a new router plus one line:

backend/main.py

```
from routers import clothing, outfits, stats

app.include_router(clothing.router)
app.include_router(outfits.router)
app.include_router(stats.router)
```

Cohesion

`Sidebar.jsx` only renders navigation. `Button.jsx` only renders a styled button and knows nothing about what happens on click. `analytics_service.py` only calculates figures and performs no I/O. `routers/clothing.py` only

handles HTTP concerns such as 404s and status codes. Most files are under sixty lines, and each one can be read and tested on its own.

Low coupling

Modules communicate only through explicit interfaces: React props, the single `api.js` client, the Pydantic schemas, and the repository interface. `Layout` renders pages through React Router's `<Outlet />`, so the shell never imports a page and pages never import the sidebar:

frontend/src/components/Layout.jsx

```
 * so the shell never imports a page and pages never import the sidebar.
 */
export default function Layout() {
  return (
    <div className="layout">
      <Sidebar />
      <div className="main">
        <Outlet />
      </div>
    </div>
  );
}
```

3.5 User interface design

The six screens were designed in Figma and then implemented as six React pages with the same names. The brand mark returns to the Dashboard from anywhere, and the account chip opens a menu holding profile, settings and log out. They share one design system: royal blue #3B6FF6 for primary actions and the active navigation item, white cards with rounded corners and soft shadows on a light grey #F6F7F9 ground, and a single sans-serif typeface.

Decision	Effect on usability
One fixed sidebar on every screen after login	Navigation never moves, so the layout is learned once
Exactly one blue primary button per screen	The next step is always obvious; everything else is secondary
The same card used for items, outfit slots and statistics	Content is recognised instantly across screens
Search and filters sit directly above the wardrobe grid	Supports the “find an item in seconds” objective
Short labelled forms with placeholder examples	Reduces the effort of the one tedious task — data entry
Explicit loading and error messages	The user is never left looking at a blank screen
Garments drawn from category and colour when no photo exists	The wardrobe looks complete from the first item

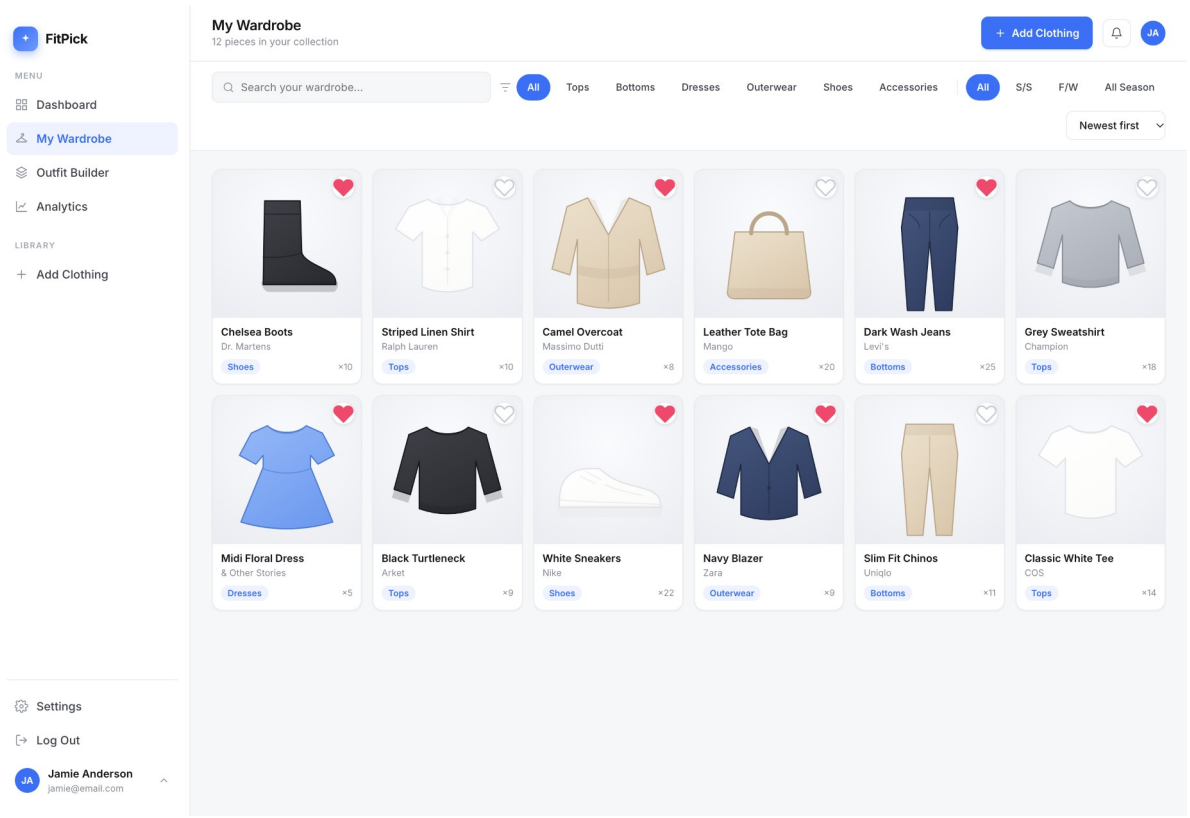


Figure 2 — My Wardrobe: the design system applied. The other five screens are in Part A, Section 8.

3.6 Key design decisions

Decision	Alternative considered	Why this one
Layered 3-tier architecture	Micro-services	One developer, one database and a CRUD workload; micro-services would add deployment cost and no benefit
Repository interface with two implementations	Calling SQL directly from routers	The test suite runs with no database, and the storage engine can change without touching an endpoint
Separate analytics service	Calculations inside routers	Keeps calculations testable without HTTP and keeps routers cohesive
Pydantic schemas as the API contract	Hand-written validation	Validation, serialisation and the OpenAPI documentation all come from one definition
Normalised PostgreSQL schema with a join table	Storing outfit items as a JSON array	An outfit-to-item link is genuinely many-to-many; foreign keys keep it consistent
SVG garments generated from category and colour	Bundled stock photographs	No licensing or storage concern, consistent look, and real photo upload can replace it later
Hand-drawn SVG charts	A charting library	Two chart shapes did not justify roughly 100 kB of dependency
Docker Compose for the whole stack	Manual local installation	One command reproduces the exact environment on any machine

4. Design of tests

Tests were designed from the acceptance criteria of the user stories, not written afterwards to match the code. Three levels are used, and a fourth technique — mutation testing — measures whether those tests are actually strong. The full run output is in Part A; this section explains how the cases were **designed**.

4.1 Test levels and their purpose

Level	What it proves	Tests
Unit	One calculation is correct in isolation, including its boundaries	11
Integration	Router, validation, serialisation and storage work together over real HTTP	25
Regression	A fault that was found once cannot return	10
Mutation	The tests above would actually fail if the code were wrong	30 mutants

4.2 Test-case design techniques

Cases were derived using standard black-box techniques rather than chosen arbitrarily:

Technique	Applied to	Cases produced
Equivalence partitioning	The category field: valid categories vs. anything else	One valid category is accepted; “Hats” is rejected with HTTP 422
Boundary value analysis	“Added this week”, which uses a seven-day cut-off	Items 0, 7 and 8 days old; only the first two count
Boundary value analysis	Rounding of wardrobe value and average wears	Values chosen to produce a third decimal place and a repeating decimal
State transition testing	The favourite flag on an item	Off → on → off must return to the original value
Error guessing	A brand-new account with nothing in it	Empty wardrobe must not divide by zero or call max() on an empty list
Interface testing	Every REST endpoint	Correct status codes: 200, 201, 204, 404, 422

4.3 Representative test cases

ID	Pre-condition	Action	Expected result
TC-01	Seeded wardrobe of 12 items	POST /clothing with a valid body	201 Created; the item is returned with a new id and appears in the list
TC-02	Seeded wardrobe	POST /clothing with category “Hats”	422 Unprocessable Entity; nothing is stored
TC-03	Seeded wardrobe	GET /clothing?search=levi	Only “Dark Wash Jeans” is returned (brand match)
TC-04	Seeded wardrobe	GET /clothing?category=Shoes	Exactly the two Shoes items are returned
TC-05	Item 5 is not a favourite	POST /clothing/5/favourite twice	is_favourite returns to its original value
TC-06	No item with id 9999	DELETE /clothing/9999	404 Not Found
TC-07	Seeded wardrobe	GET /stats/dashboard	total_items equals the number of items; the breakdown sums to the same number
TC-08	Empty wardrobe	analytics_stats(repository)	average_wears is 0.0 and top_category is the placeholder, with no exception
TC-09	Items 0, 7 and 8 days old	dashboard_stats(...)	added_this_week is 2
TC-10	Outfit does not exist	POST /outfits with item_ids = []	422 Unprocessable Entity

4.4 Test data strategy

A twelve-item seed wardrobe is defined once in `backend/seed_data.py` and mirrored in `database/seed.sql`, so the tests, the development database and the screenshots all describe the same wardrobe. Tests that need a different shape of data build their own: the `empty_wardrobe` fixture supplies an unseeded repository pair for edge-case work.

backend/tests/test_analytics_unit.py — a helper builds precise data for boundary cases

```
    item = repo.add(ClothingItemCreate(
        name=name, category=category, colour=colour, brand=brand, price=price
    ))
    item.times_worn = worn
    item.created_at = date.today() - timedelta(days=days_old)
    return item
```

4.5 Traceability: user story to test

Every Must Have story with implemented behaviour is covered by at least one automated test:

Story	Covering tests
4 Add clothing item	test_add_item_then_read_it_back, test_add_item_increases_wardrobe_count, test_rejects_unknown_category, test_rejects_item_without_a_name
5 Edit clothing item	test_update_item_changes_only_given_fields, test_reg07_updating_one_field_keeps_the_others
6 Delete clothing item	test_delete_item_removes_it, test_reg02_deleting_a_missing_item_is_a_404
7 View wardrobe	test_seeded_wardrobe_is_returned
8 Search clothing	test_search_matches_item_name, test_reg01_search_also_matches_the_brand, test_reg08_search_is_case_insensitive
9 Filter clothing	test_filter_by_category, test_reg03_category_all_does_not_filter_anything_out
10 Create outfit	test_create_outfit_from_existing_items, test_outfit_needs_at_least_one_item, test_outfit_rejects_unknown_item
12 Delete outfit	test_delete_outfit
13 Favourite outfit	test_favourite_an_outfit
14 View dashboard	test_dashboard_totals_match_the_wardrobe, test_breakdown_adds_up_to_the_total, test_reg10_deleted_item_disappears_from_the_statistics
19 Wardrobe statistics	test_analytics_wardrobe_value_is_the_sum_of_prices, test_average_wears_is_the_mean_rounded_to_one_decimal, plus 6 more unit tests
21 Favourite clothing items	test_toggle_favourite_flips_the_flag, test_reg06_favourite_toggles_back_to_its_original_value
23 Sort clothing items	test_sort_by_most_worn

4.6 Results

Test outcome

46 of 46 tests pass.
Mutation score 100% (30 of 30 mutants killed), improved from 60% after eleven mutation-driven unit tests were added.
Frontend: ESLint clean and the production build succeeds.
Stories 1–3, 15, 20 and 25 are not covered by automated tests because their backend does not exist yet; they are listed as future work rather than claimed as complete.

5. Appendix A — Software Requirements Specification

A.1 Purpose

This appendix specifies what the FitPick software must do. It is written for the developer and the assessor, and is the reference against which the tests in Section 4 were designed.

A.2 Product perspective

FitPick is a self-contained web application with no dependency on an external system. It consists of a browser client, a REST API and a relational database, deployed together as three containers. It replaces no existing system; it fills a gap between a shopping app and a spreadsheet.

A.3 Functional requirements

ID	Requirement	Priority	Status
FR-01	The system shall allow a user to record a clothing item with name, category, subcategory, colour, season, brand, size, price, tags and notes.	Must	Done
FR-02	The system shall reject a clothing item whose category is not one of the six defined categories.	Must	Done
FR-03	The system shall reject a clothing item with no name or no colour.	Must	Done
FR-04	The system shall list all clothing items belonging to the user.	Must	Done
FR-05	The system shall filter clothing items by free-text search over name and brand.	Must	Done
FR-06	The system shall filter clothing items by category and by season.	Must	Done
FR-07	The system shall sort clothing items by newest, name, most worn or price.	Should	Done
FR-08	The system shall update any subset of a clothing item's fields, leaving the others unchanged.	Must	Done
FR-09	The system shall delete a clothing item, and return 404 if it does not exist.	Must	Done
FR-10	The system shall toggle the favourite flag on a clothing item.	Should	Done
FR-11	The system shall create an outfit from one or more existing clothing items.	Must	Done
FR-12	The system shall reject an outfit with no items or with an unknown item id.	Must	Done
FR-13	The system shall list saved outfits and allow one to be reopened for editing.	Must	Partial
FR-14	The system shall delete an outfit and toggle its favourite flag.	Must	Done
FR-15	The system shall report total items, total outfits, favourites, most-worn item, items added in the last seven days and a per-category breakdown.	Must	Done
FR-16	The system shall report wardrobe value, average wears per item, top category, favourite brand and a colour breakdown.	Should	Done
FR-17	The system shall allow a user to register, log in and log out.	Must	Future
FR-18	The system shall allow a user to update profile details and change password.	Must	Future
FR-19	The system shall store an uploaded photograph for a clothing item.	Should	Future

A.4 Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	A wardrobe query shall return in under one second for a wardrobe of 500 items.
NFR-02	Usability	A new user shall be able to record their first item within two minutes.
NFR-03	Portability	The system shall build and run on any machine with Docker, using one command.
NFR-04	Maintainability	No module shall depend on the internals of another; all communication is through defined interfaces.
NFR-05	Testability	The automated test suite shall run without a database and complete in under five seconds.
NFR-06	Reliability	The system shall behave correctly for an empty wardrobe and for missing records.
NFR-07	Security	Passwords shall be stored as hashes; the schema provides password_hash rather than a plain field.
NFR-08	Compatibility	The interface shall work in current versions of Chrome, Safari, Firefox and Edge.

A.5 External interfaces

Interface	Description
User interface	Six browser screens served by Vite on port 5173, described in Section 3.5
API interface	A JSON REST API on port 8000; the OpenAPI description is generated at /docs
Database interface	PostgreSQL 16 on container port 5432, published on host port 5433, reached with psycopg
Configuration	DATABASE_URL selects PostgreSQL storage; VITE_API_URL overrides the backend address

A.6 API specification

Method and path	Purpose	Success
`GET /`	Service banner and version	200
`GET /health`	Liveness probe	200
`GET /clothing`	List items, with search, category, season and sort parameters	200
`GET /clothing/categories`	The six valid categories	200
`GET /clothing/{id}`	One item	200 / 404
`POST /clothing`	Create an item	201 / 422
`PUT /clothing/{id}`	Partially update an item	200 / 404
`POST /clothing/{id}/favourite`	Toggle the favourite flag	200 / 404
`DELETE /clothing/{id}`	Delete an item	204 / 404
`GET /outfits`	List saved outfits	200
`GET /outfits/occasions`	The six valid occasions	200
`POST /outfits`	Create an outfit	201 / 422
`POST /outfits/{id}/favourite`	Toggle the favourite flag	200 / 404
`DELETE /outfits/{id}`	Delete an outfit	204 / 404
`GET /stats/dashboard`	Dashboard figures	200
`GET /stats/analytics`	Analytics figures	200

6. Appendix B — Data Flow Diagrams

The data flow diagrams describe FitPick in terms of the data that moves through it. The **context diagram** shows the system as a single process with its one external entity; the **level 1 diagram** decomposes that process into the five activities the system performs and the four stores it keeps.

B.1 Context diagram (Level 0)

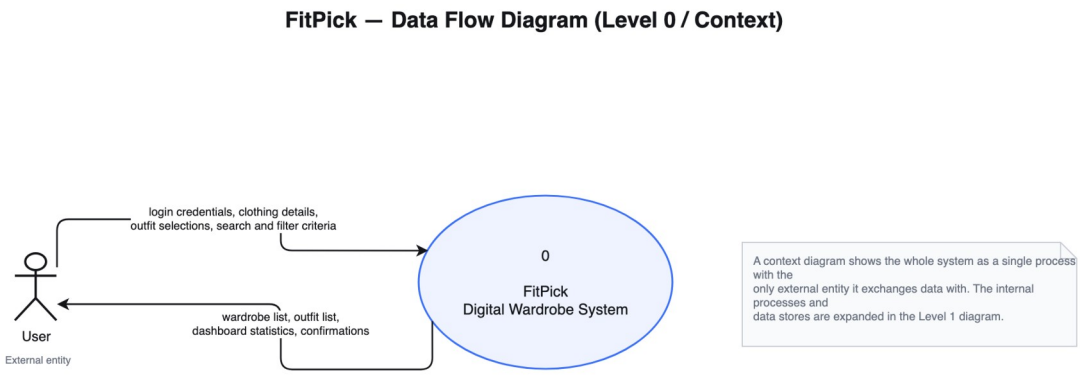


Figure 3 — Level 0 context diagram

B.2 Level 1 diagram

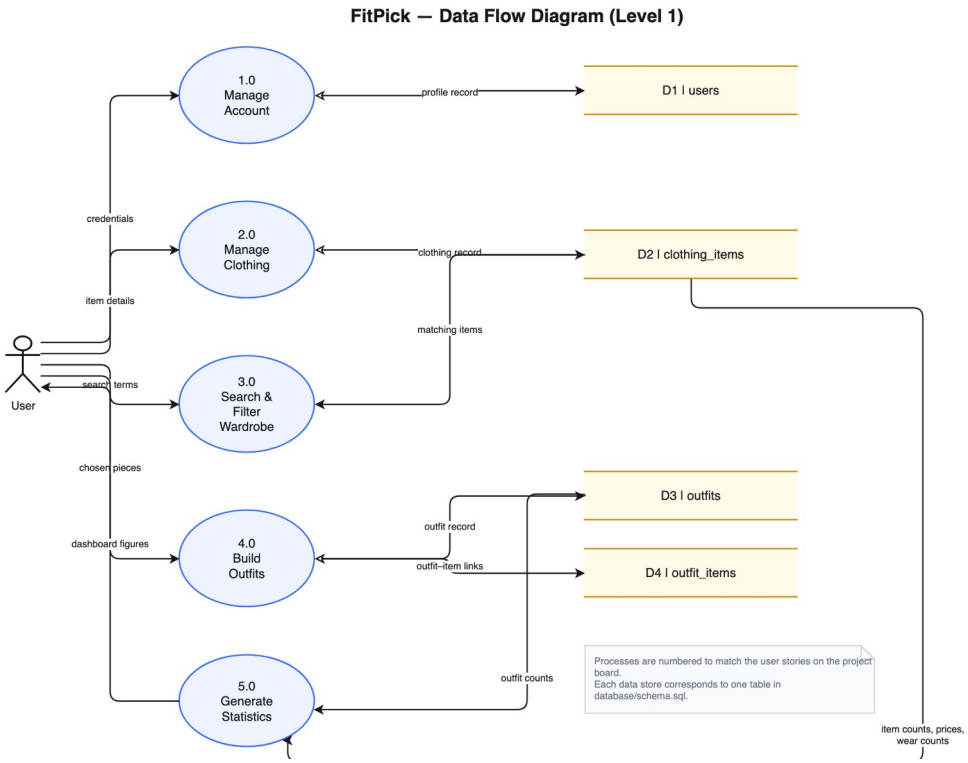


Figure 4 — Level 1 data flow diagram. Each data store maps to one table in database/schema.sql.

B.3 Process descriptions

Process	Input	Processing	Output
1.0 Manage Account	Credentials, profile details	Validate and persist the user record	Confirmation; profile record in D1
2.0 Manage Clothing	Item details, item id	Validate against the schema, then create, update or delete	Clothing record in D2; confirmation
3.0 Search & Filter Wardrobe	Search text, category, season, sort order	Read D2 and apply the filters and ordering in turn	The matching list of items
4.0 Build Outfits	Chosen item ids, name, occasion	Check every item exists, then write the outfit and its links	Outfit in D3; links in D4
5.0 Generate Statistics	Item and outfit records	Count, sum, average and rank	Dashboard and analytics figures

7. Appendix C — Entity Relationship Diagram

FitPick stores four entities. A user owns many clothing items and creates many outfits; an outfit contains many clothing items and a clothing item can appear in many outfits, so that relationship is resolved by the `outfit_items` join table.

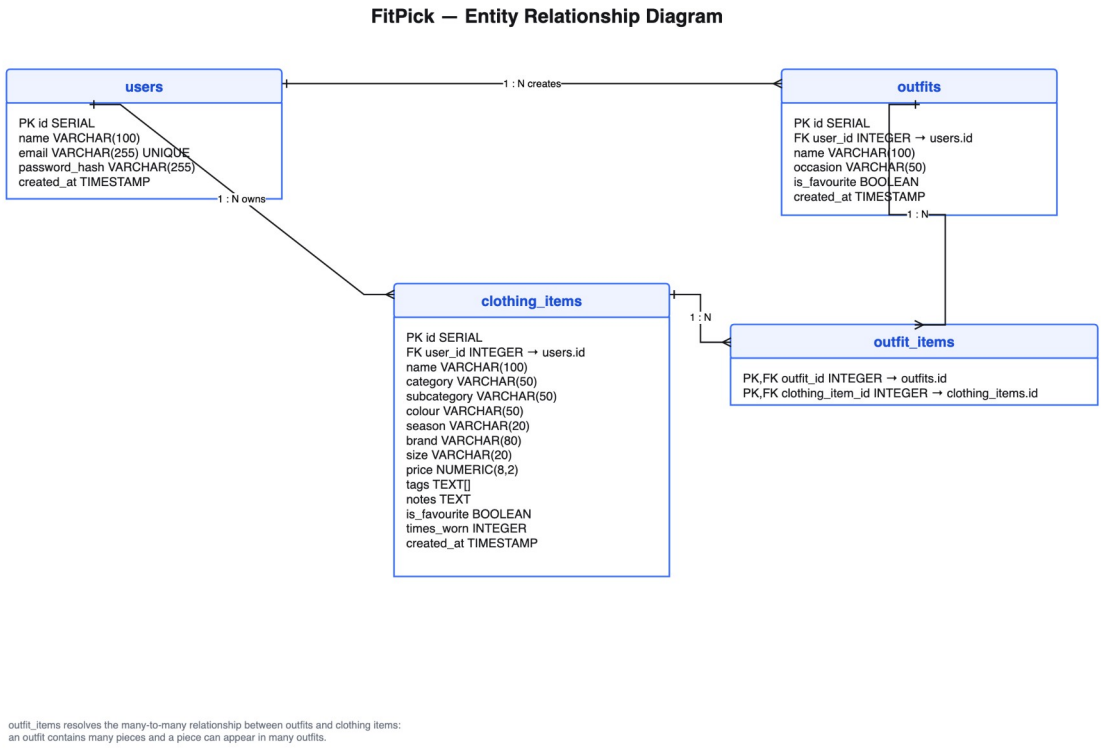


Figure 5 — Entity relationship diagram (crow's-foot notation)

C.1 Data dictionary — clothing_items

Column	Type	Constraint	Meaning
`id`	SERIAL	Primary key	Surrogate identifier
`user_id`	INTEGER	FK → users.id, NOT NULL	Owner of the garment
`name`	VARCHAR(100)	NOT NULL	Display name, e.g. “Navy Blazer”
`category`	VARCHAR(50)	NOT NULL	One of six: Tops, Bottoms, Outerwear, Shoes, Dresses, Accessories
`subcategory`	VARCHAR(50)	Default ''	Finer type, e.g. Blazer, Jeans, Sneakers
`colour`	VARCHAR(50)	NOT NULL	Primary colour; also selects the garment illustration tint
`season`	VARCHAR(20)	Default 'All Season'	S/S, F/W or All Season
`brand`	VARCHAR(80)	Default ''	Used by search and by the favourite-brand statistic
`size`	VARCHAR(20)	Default ''	Free text, because sizing differs by brand
`price`	NUMERIC(8,2)	Default 0	Price paid; feeds the wardrobe-value statistic
`tags`	TEXT[]	Default '{}'	Occasion tags such as Casual or Office
`notes`	TEXT	Default ''	Free text, e.g. care instructions
`image_url`	TEXT	Nullable	Reserved for the photo-upload story
`is_favourite`	BOOLEAN	Default FALSE	Marked by the heart control in the wardrobe grid
`times_worn`	INTEGER	Default 0	Wear counter; feeds most-worn and average-wears
`created_at`	TIMESTAMP	Default NOW()	Used by “recently added” and “added this week”

C.2 Other tables

Table	Columns	Notes
users	id, name, email (unique), password_hash, created_at	A password hash is stored, never a plain password
outfits	id, user_id, name, occasion, is_favourite, created_at	Occasion is one of six values used by the Outfit Builder
outfit_items	outfit_id, clothing_item_id	Composite primary key; both columns cascade on delete

C.3 Referential integrity

Every foreign key uses `ON DELETE CASCADE`, so deleting a user removes their wardrobe and outfits, and deleting a garment removes it from every outfit that referenced it. This prevents orphan rows without any application-level clean-up code.

database/schema.sql — the join table that resolves the many-to-many relationship

```
-- Join table: which clothing items make up an outfit (many-to-many)
CREATE TABLE IF NOT EXISTS outfit_items (
    outfit_id        INTEGER NOT NULL REFERENCES outfits(id) ON DELETE CASCADE,
    clothing_item_id INTEGER NOT NULL REFERENCES clothing_items(id) ON DELETE CASCADE,
    PRIMARY KEY (outfit_id, clothing_item_id)
);

CREATE INDEX IF NOT EXISTS idx_clothing_user     ON clothing_items (user_id);
CREATE INDEX IF NOT EXISTS idx_clothing_category ON clothing_items (category);
CREATE INDEX IF NOT EXISTS idx_outfits_user      ON outfits (user_id);
```

8. Appendix D — UML diagrams

D.1 Use case diagram

Two actors interact with FitPick. A **Guest** can only register or log in. A **Registered User** reaches everything else. The grouped use cases on the left are the ones a user starts directly; each includes the detailed sub-functions on the right.

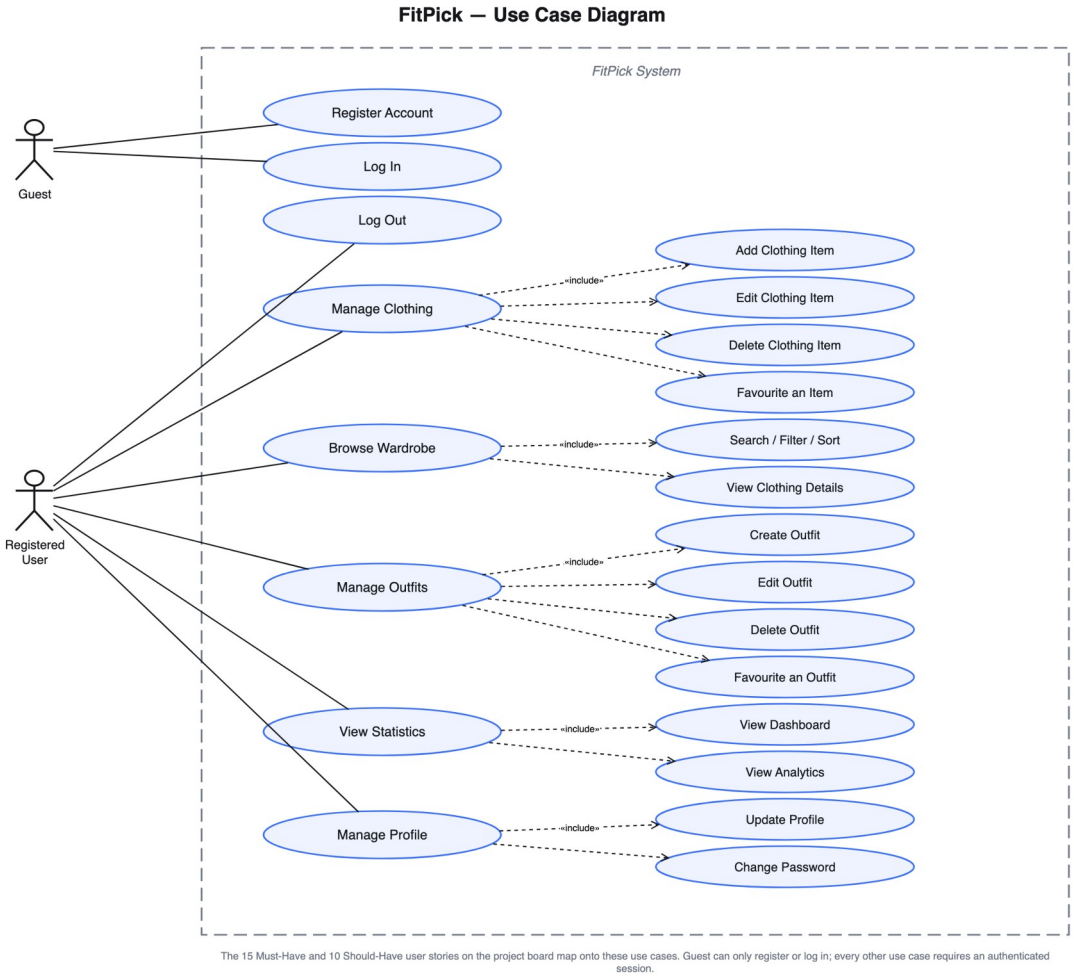


Figure 6 — Use case diagram. The 25 backlog stories map onto these use cases.

D.2 Class diagram

The backend class diagram shows the structure that makes the storage layer replaceable. `ClothingRepository` is an abstract interface; two concrete classes implement it, and the routers and the analytics service depend on the interface alone.

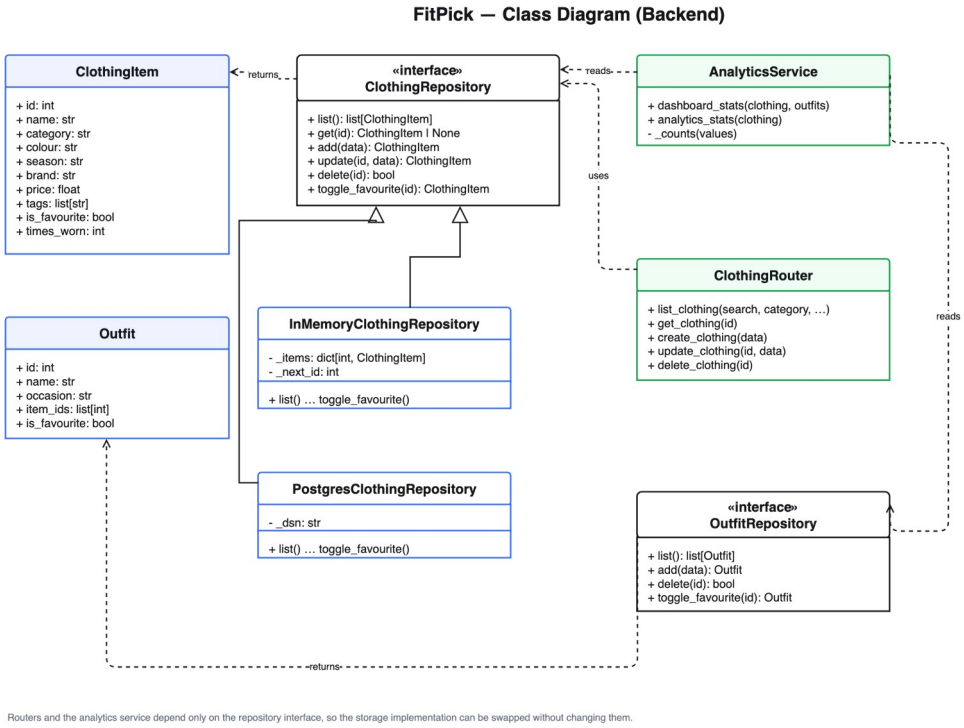


Figure 7 — Class diagram of the backend

D.3 Sequence diagram

Adding a clothing item is the fullest path through the system — it touches every tier and has both a success and a failure branch:

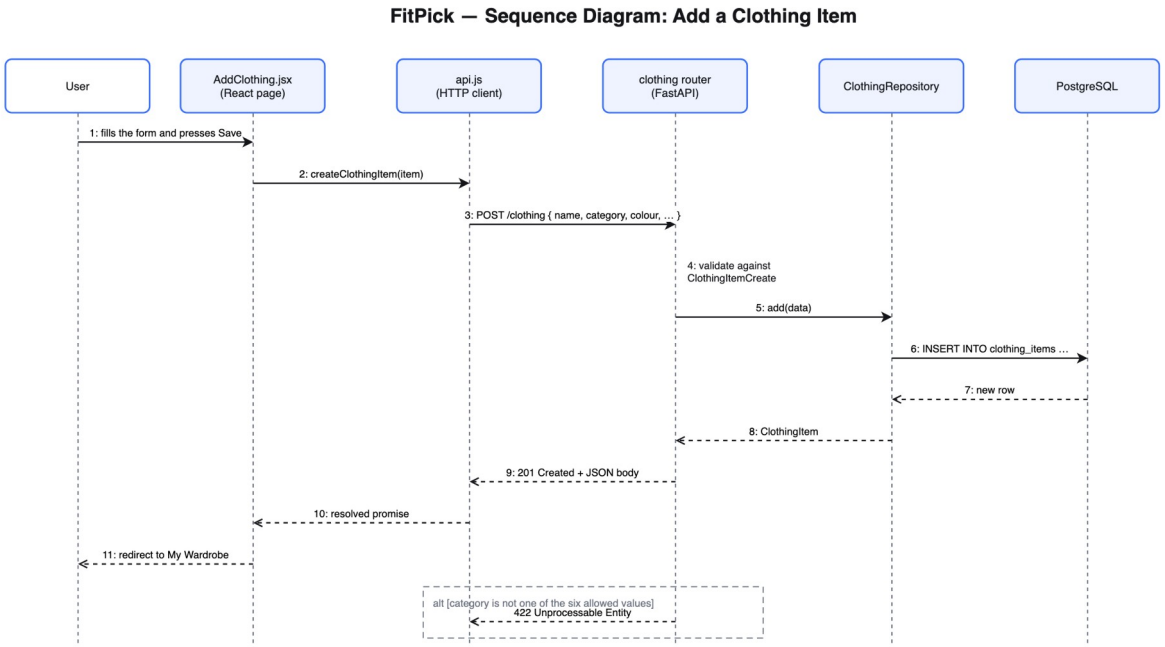


Figure 8 — Sequence diagram: adding a clothing item

9. Appendix E — Code listing and GitHub link

E.1 Repository

Source code

https://github.com/nithi0307/fitpick

The repository contains the full source, the automated tests, the editable Draw.io diagram sources, the exported PNGs, every screenshot used in these reports, and both report documents.

E.2 Repository structure

```
fitpick/
├── frontend/                        React 19 + Vite single-page application
│   └── src/
│       ├── App.jsx                  route table
│       ├── pages/                   Login, Dashboard, Wardrobe, AddClothing,
│       │                            OutfitBuilder, Profile
│       ├── components/              Layout, Sidebar, Topbar, Button, Card,
│       │                            ItemCard, Charts, Garment, Icon, HeroArt
│       ├── services/api.js          the only module that calls the backend
│       └── styles/global.css        design tokens and shared component styles
├── backend/                         FastAPI application
│   ├── main.py                      app factory: CORS + router registration
│   ├── routers/                     clothing.py, outfits.py, stats.py
│   ├── schemas.py                   Pydantic models — the API contract
│   ├── services/analytics_service.py dashboard and analytics calculations
│   ├── repositories/                base.py, memory_repository.py,
│   │                                postgres_repository.py, __init__.py (factory)
│   ├── seed_data.py                 the twelve-item sample wardrobe
│   └── tests/                       46 tests across unit, integration, regression
├── database/
│   ├── schema.sql                   four tables, foreign keys and indexes
│   └── seed.sql                     the same sample wardrobe, in SQL
├── docs/
│   ├── design/architecture/         architecture .drawio + .png
│   ├── design/diagrams/             DFD, ERD, use case, class, sequence
│   ├── design/wireframes/           the six designed screens
│   ├── screenshots/                 application and terminal evidence
│   └── *.pdf                        this report and the testing report
├── docker-compose.yml               frontend 5173, backend 8000, database 5433
└── README.md
```

E.3 Key listings

E.3.1 Repository interface — backend/repositories/base.py

```
from schemas import ClothingItem, ClothingItemCreate, ClothingItemUpdate, Outfit, OutfitCreate


class ClothingRepository(ABC):
    @abstractmethod
    def list(self) -> list[ClothingItem]: ...

    @abstractmethod
    def get(self, item_id: int) -> ClothingItem | None: ...

    @abstractmethod
    def add(self, data: ClothingItemCreate) -> ClothingItem: ...

    @abstractmethod
    def update(self, item_id: int, data: ClothingItemUpdate) -> ClothingItem | None: ...

    @abstractmethod
    def delete(self, item_id: int) -> bool: ...

    @abstractmethod
    def toggle_favourite(self, item_id: int) -> ClothingItem | None: ...
```

E.3.2 Analytics service — backend/services/analytics_service.py

```
def _counts(values: list[str]) -> list[CategoryCount]:
    counter = Counter(values)
    ordered = sorted(counter.items(), key=lambda pair: (-pair[1], pair[0]))
    return [CategoryCount(label=label, count=count) for label, count in ordered]



def dashboard_stats(clothing: ClothingRepository, outfits: OutfitRepository) -> DashboardStats:
    items = clothing.list()
    cutoff = date.today() - timedelta(days=7)
    recent = [i for i in items if i.created_at and i.created_at >= cutoff]
    most_worn = max(items, key=lambda i: i.times_worn, default=None)

    return DashboardStats(
        total_items=len(items),
        total_outfits=len(outfits.list()),
        favourites=sum(1 for i in items if i.is_favourite),
        most_worn=most_worn.name if most_worn else "—",
        most_worn_count=most_worn.times_worn if most_worn else 0,
        added_this_week=len(recent),
        breakdown=_counts([i.category for i in items]),
    )



def analytics_stats(clothing: ClothingRepository) -> AnalyticsStats:
    items = clothing.list()
    categories = _counts([i.category for i in items])
    brands = _counts([i.brand for i in items if i.brand])
```

E.3.3 Clothing router — backend/routers/clothing.py

```


router = APIRouter(prefix="/clothing", tags=["clothing"])



@router.get("", response_model=list[ClothingItem])
def list_clothing(
    search: str = "",
    category: str = "All",
    season: str = "All",
    sort: str = "newest",
    repository: ClothingRepository = Depends(get_clothing_repository),
):
    items = repository.list()

    if search:
        needle = search.lower()
        items = [i for i in items if needle in i.name.lower() or needle in i.brand.lower()]
    if category != "All":
        items = [i for i in items if i.category == category]
    if season != "All":
        items = [i for i in items if i.season == season]

    sorters = {
        "newest": lambda i: -i.id,
        "name": lambda i: i.name.lower(),
        "most-worn": lambda i: -i.times_worn,
        "price": lambda i: -i.price,
    }
    return sorted(items, key=sorters.get(sort, sorters["newest"]))



@router.get("/categories", response_model=list[str])
def list_categories():
```

E.3.4 API client — frontend/src/services/api.js

```
const BASE_URL = import.meta.env.VITE_API_URL ?? "http://localhost:8000";

async function request(path, options = {}) {
  const response = await fetch(`${BASE_URL}${path}`, {
    headers: { "Content-Type": "application/json" },
    ...options,
  });

  if (!response.ok) {
    throw new Error(`Request failed: ${response.status}`);
  }

  return response.status === 204 ? null : response.json();
}

function query(params) {
  const search = new URLSearchParams(
    Object.entries(params).filter(([, value]) => value !== "" && value != null)
  );
  return search.toString() ? `?${search}` : "";
}
```

E.3.5 Route table — frontend/src/App.jsx

```
 * Route table. Login stands alone; every other screen is nested inside
 * Layout and therefore gets the sidebar automatically.
 */
export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Login />} />

        <Route element={<Layout />}>
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/wardrobe" element={<Wardrobe />} />
          <Route path="/add-clothing" element={<AddClothing />} />
          <Route path="/outfit-builder" element={<OutfitBuilder />} />
          <Route path="/profile" element={<Profile />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

E.3.6 Wardrobe screen — frontend/src/pages/Wardrobe.jsx (extract)

```
  const [status, setStatus] = useState("loading");
  const [search, setSearch] = useState("");
  const [category, setCategory] = useState("All");
  const [season, setSeason] = useState("All");
  const [sort, setSort] = useState("newest");

  useEffect(() => {
    let cancelled = false;
    getClothingItems({ search, category, season, sort })
      .then((data) => {
        if (!cancelled) {
          setItems(data);
          setStatus("ready");
        }
      })
      .catch(() => !cancelled && setStatus("error"));
    return () => {
      cancelled = true;
    };
  }, [search, category, season, sort]);

  async function handleFavourite(item) {
    const updated = await toggleClothingFavourite(item.id);
    setItems((current) => current.map((i) => (i.id === updated.id ? updated : i)));
  }

  return (
    <>
      <Topbar title="My Wardrobe" subtitle={`${items.length} pieces in your collection`}>
        <Link to="/add-clothing">
          <Button>
```

E.4 Running the project

With Docker — the whole system

```
$ git clone https://github.com/nithi0307/fitpick.git
$ cd fitpick
$ docker compose up --build
```

Without Docker — for development

```
# terminal 1
$ cd backend && python3.13 -m venv .venv
$ .venv/bin/pip install -r requirements-dev.txt
$ .venv/bin/uvicorn main:app --reload

# terminal 2
$ cd frontend && npm install && npm run dev
```

E.5 Future work

- Real authentication: password hashing, sessions and per-user data, which the schema already supports through user_id.
- Screens for editing and deleting clothing items and outfits, so stories 5, 6 and 12 become end-to-end rather than API-only.
- Photograph upload and storage, replacing the generated garment illustrations.
- A wear-logging action so times_worn is updated by the user rather than seeded.
- Outfit suggestions based on occasion, season and what has not been worn recently.